

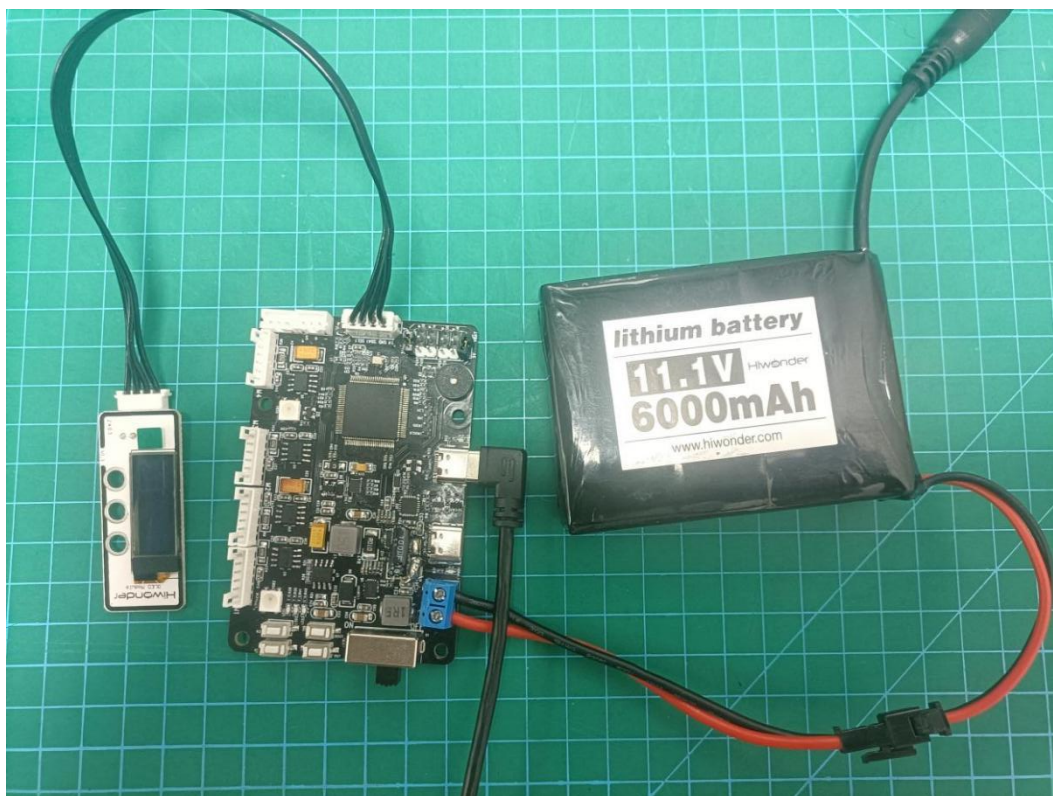
Lesson 4 RRCLite ADC Power Supply Voltage Detection

1. Program Description

This section primarily involves using the built-in ADC circuit in the STM32 main control board to read the battery voltage and output the voltage value via the serial port and OLED.

2. Hardware Connection

Connect the RRCLite controller to the PC via USB cable through serial port 1, and connect it to the OLED through the I2C interface. The hardware connection diagram is as follows:



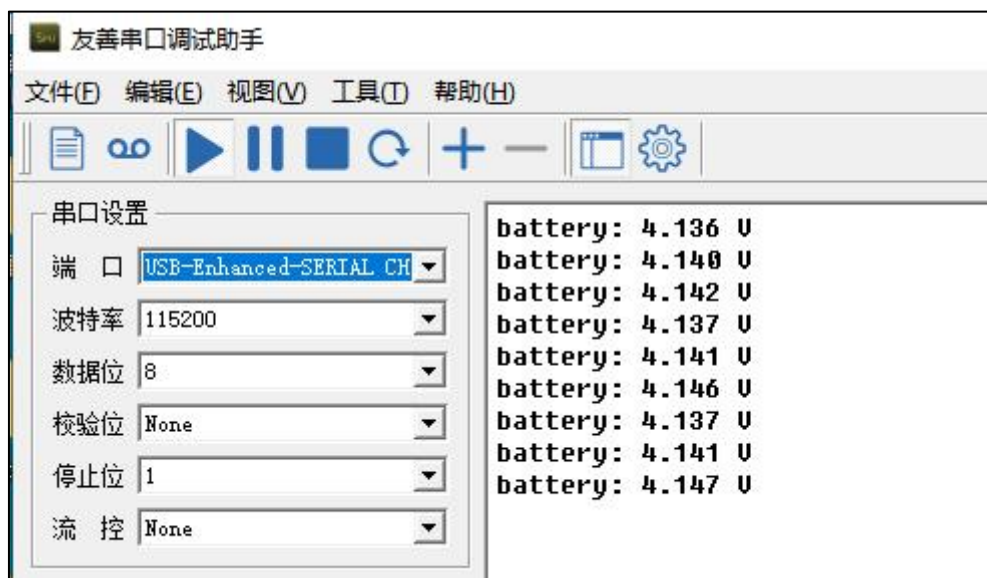
3. Program Download

The example for this section is "RosRobotControllerLite_ros".

For specific download methods, refer to the document: "2. STM32 Development Fundamentals/3. Project Compilation and Download".

4. Program Outcome

1) Serial port



(Note: The development board needs to be powered by PC with the USB cable)

Open the serial assistant (located in "Appendix/3.Software/Serial Assistant").

Find the serial port connected to the development board. It may not necessarily be COM4. Please select it based on actual conditions. Set it as follows:

Baud Rate: 115200

Data Bits: 8

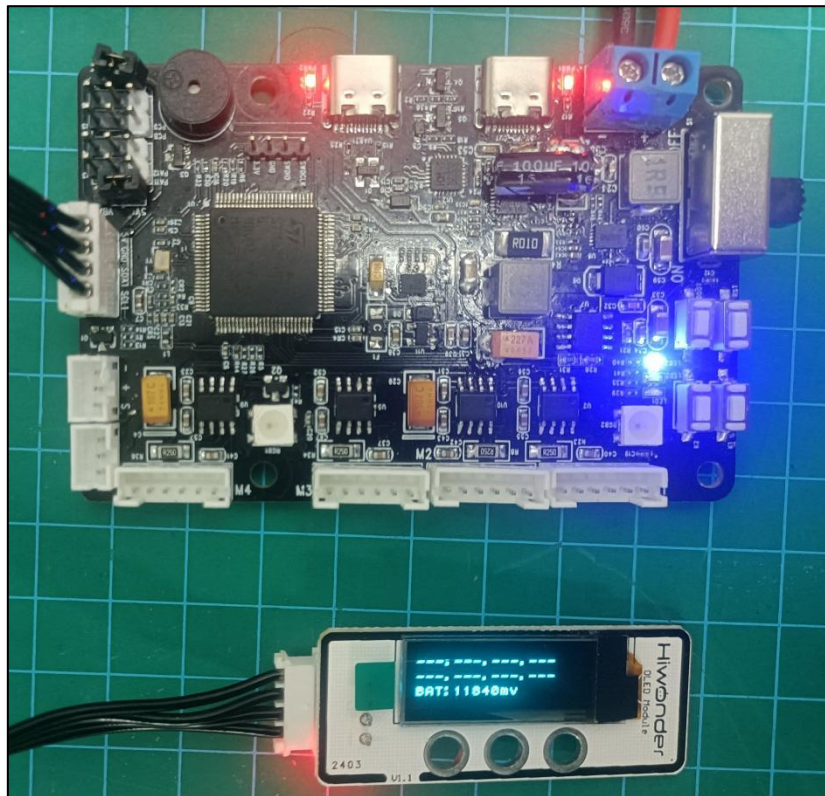
Parity: None

Stop Bits: 1

Display Encoding Format: UTF-8

After opening the serial port, you will see continuously refreshed data, where the data following "battery" is the battery voltage.

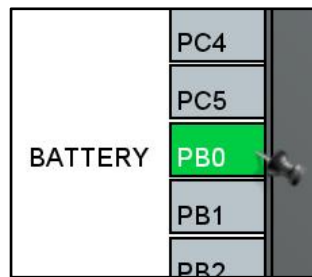
2) OLED Display



(Note: The development board needs to be powered by the battery)

5. Pin Configuration

Use the STM32CubeMX software to view pin information. During program operation, the ADC conversion function of the pin will be enabled. The corresponding chip pin is: PB0



6. Source Code Analysis

The source code for this course is located in the same folder as this tutorial document, mainly analyzing the Hiwonder/System/app.c and Hiwonder/System/battery_handle.c files.

6.1 Voltage Print Output

```

1  #include "app.h"
2  /**
3   * @file app.c
4   * @author Lu Yongping (Lucas@hiwonder.com)
5   * @brief main application logic
6   * @version 0.1
7   * @date 2023-05-08
8   * @copyright Copyright (c) 2023
9   */
10
11 #include "cmsis_os2.h"
12 #include "led.h"
13 #include "lwmem_porting.h"
14 #include "global.h"
15 #include "global.h"
16 #include "adc.h"
17 #include "u8g2_porting.h"
18 #include "packet_reports.h"
19 #include "packet_handle.h"
20 #include "serial_servo.h"
21 #include "rgb_spi.h"
22
23 void buzzers_init(void);
24 void buttons_init(void);
25 void leds_init(void);
26 void motors_init(void);
27 void serial_servo_init(void);
28 void pwm_servos_init(void);
29 void chassis_init(void);
30
31

```

The App.c file includes the user entry function app_task_entry(). The main programming flow is as follows:

- Start the battery monitoring timer, which detects once every 50ms.
- Then start the ADC conversion channel.
- Finally, continuously print the battery voltage value.

The external variable battery_volt stores the measured battery voltage value.

Therefore, by reading this value, you can obtain the voltage.

6.2 Reading Battery Voltage

```

1 #include "adc.h"
2 #include "global.h"
3 #include "global_conf.h"
4 #include "buzzer.h"
5 #include "packet_reports.h"
6 #include "packet.h"
7
8 float battery_volt = 0.0f; /* Global variable of battery voltage. unit v */
9 static uint16_t battery_min_limit = 6300; /* Low voltage alarm value */
10
11 static uint16_t adc_value[2];
12
13 extern osMessageQueueId_t lvgl_event_queueHandle;
14
15 void battery_check_timer_callback(void *argument)
16 {
17     if(adc_value[0] != 0 && adc_value[0] != 4095) { /* The internal reference voltage cannot be 0, otherwise it cannot be calculated */
18         //float vdda = 3300.0f * ((float)((C_IO_uint16_t)(0x1FFF7A2A))) / ((float)adc_value[0]);
19         //float volt = vdda / 4095.0f * ((float)adc_value[0]) * 11.0f; /* 100k+10k resistor voltage division, the actual voltage is 11 times the measured voltage */
20         float volt = 1210.0f / ((float)adc_value[0]) * ((float)adc_value[1]) * 11.0f;
21         volt = volt > 20000 ? 0 : volt; /* ADC reading value exceeds the maximum allowable power supply voltage, data error */
22         battery_volt = battery_volt == 0 ? volt : battery_volt * 0.95f + volt * 0.05f;
23     }
24 }

```

The battery_handle.c file contains the battery_check_timer_callback() function, which is the callback function for the battery monitoring timer and is entered every 50ms to read the voltage value. The ADC value read will be stored in the adc_value array. The program flow is as follows:

First, check if adc_value[] is an error read value.

If the read value is correct, calculate and convert the value to a voltage value and store it in the global variable battery_volt.

If incorrect, do nothing.

Start a new ADC value reading.

The HAL_ADC_Start_DMA() function enables the DMA method for ADC reading:

```

HAL_StatusTypeDef HAL_ADC_Start_DMA ( ADC_HandleTypeDef* hadc,
uint32_t* pData ,  uint32_t Length )

```

Parameter 1: ADC handle

Parameter 2: Storage address of the ADC value

Parameter 3: Size of the ADC data

This function allows the ADC value to be automatically stored in the storage address without needing the CPU.

```
#if ENABLE_OLED
    extern int oled_battery;
    oled_battery = (int) (battery_volt + 0.5f);
#endif
```

The collected voltage data will be stored in an external variable, which will be printed on the OLED screen in its processing function.